Name of Student: Postiwala Vatsal Dipakbhai	PRN: 26070122509
Subject: Web Application and Development	Sem: III AY: 2026-27
EXPERIMENT No: 10	Assignment No: 10
TITLE: To write operational database scripts executing basic atomic CRUD transactions (Create, Read, Update, Delete) on specialized user collections.	

Objective

To perform CRUD (Create, Read, Update, Delete) operations on a MongoDB collection using mongosh.

Theory

CRUD operations form the core of interacting with any database. In MongoDB: Create uses insertOne()/insertMany(); Read uses find()/findOne() with optional query filters and projections; Update uses updateOne()/updateMany() with update operators like $set, $inc, $unset; Delete uses deleteOne()/deleteMany(). MongoDB queries use a flexible JSON-like syntax supporting comparison operators ($gt, $lt, $in, etc.) and logical operators ($and, $or).

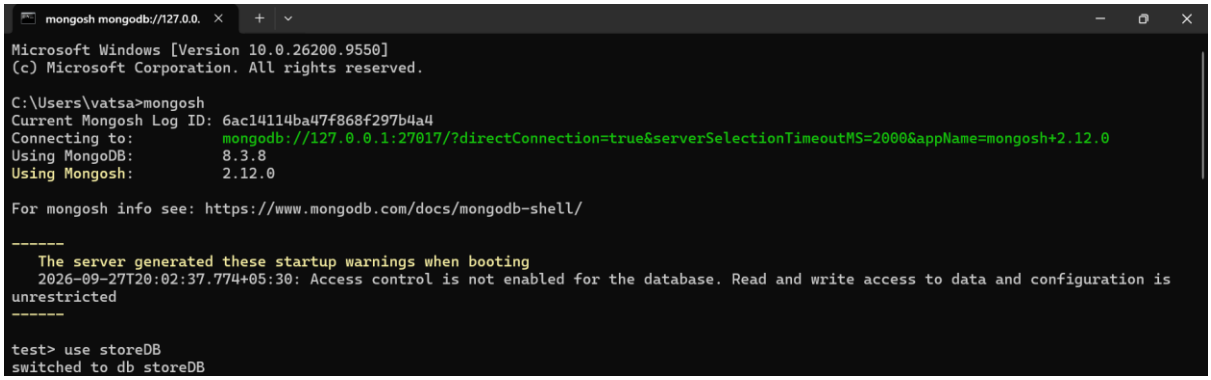
Software / Hardware Requirements

MongoDB installed and running (from Experiment 10), mongosh.

Step-by-Step Procedure

1. Open mongosh and switch to a new database named storeDB.

```
use storeDB
```



2. Create (insert) multiple product documents into a products collection using insertMany().

```
db.products.insertMany([
  { name: "Laptop", category: "Electronics", price: 55000, stock: 12 },
  { name: "Mouse", category: "Electronics", price: 500, stock: 100 },
  { name: "Desk Chair", category: "Furniture", price: 4500, stock: 20 },
  { name: "Notebook", category: "Stationery", price: 40, stock: 300 }
])
```

```
storeDB> db.products.insertMany([
|   { name: "Laptop", category: "Electronics", price: 55000, stock: 12 },
|   { name: "Mouse", category: "Electronics", price: 500, stock: 100 },
|   { name: "Desk Chair", category: "Furniture", price: 4500, stock: 20 },
|   { name: "Notebook", category: "Stationery", price: 40, stock: 300 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac14122ba47f868f297b4a5'),
    '1': ObjectId('6ac14122ba47f868f297b4a6'),
    '2': ObjectId('6ac14122ba47f868f297b4a7'),
    '3': ObjectId('6ac14122ba47f868f297b4a8')
  }
}
```

3. Read all documents using find() and pretty-print them.

```
db.products.find().pretty()
```

```
storeDB> db.products.find().pretty()
[
  {
    _id: ObjectId('6ac14122ba47f868f297b4a5'),
    name: 'Laptop',
    category: 'Electronics',
    price: 55000,
    stock: 12
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a6'),
    name: 'Mouse',
    category: 'Electronics',
    price: 500,
    stock: 100
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a7'),
    name: 'Desk Chair',
    category: 'Furniture',
    price: 4500,
    stock: 20
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a8'),
    name: 'Notebook',
    category: 'Stationery',
    price: 40,
    stock: 300
  }
]
```

4. Read with a filter: find all products in the 'Electronics' category.

```
db.products.find({ category: "Electronics" })
```

```
storeDB> db.products.find({ category: "Electronics" })
[
  {
    _id: ObjectId('6ac14122ba47f868f297b4a5'),
    name: 'Laptop',
    category: 'Electronics',
    price: 55000,
    stock: 12
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a6'),
    name: 'Mouse',
    category: 'Electronics',
    price: 500,
    stock: 100
  }
]
```

5. Use comparison operators: find all products priced above 1000.

```
db.products.find({ price: { $gt: 1000 } })
```

```
storeDB> db.products.find({ price: { $gt: 1000 } })
[
  {
    _id: ObjectId('6ac14122ba47f868f297b4a5'),
    name: 'Laptop',
    category: 'Electronics',
    price: 55000,
    stock: 12
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a7'),
    name: 'Desk Chair',
    category: 'Furniture',
    price: 4500,
    stock: 20
  }
]
```

6. Use findOne() to retrieve just a single matching document, and a projection to return only selected fields.

```
db.products.findOne({ name: "Laptop" }, { name: 1, price: 1, _id: 0 })
```

```
storeDB> db.products.findOne({ name: "Laptop" }, { name: 1, price: 1, _id: 0 })
{ name: 'Laptop', price: 55000 }
```

7. Update a single document: reduce the price of 'Mouse' by using the $set operator.

```
db.products.updateOne({ name: "Mouse" }, { $set: { price: 450 } })
```

```
storeDB> db.products.updateOne({ name: "Mouse" }, { $set: { price: 450 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

8. Update multiple documents at once: increase stock by 10 for every Electronics item using $inc.

```
db.products.updateMany({ category: "Electronics" }, { $inc: { stock: 10 } })
```

```
storeDB> db.products.updateMany({ category: "Electronics" }, { $inc: { stock: 10 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

9. Delete a single document: remove the 'Notebook' product.

```
db.products.deleteOne({ name: "Notebook" })
```

```
storeDB> db.products.deleteOne({ name: "Notebook" })
{ acknowledged: true, deletedCount: 1 }
```

10. Delete multiple documents matching a condition: remove all products with stock less than 5 (first insert one such test document, then delete it).

```
db.products.insertOne({ name: "Old Item", category: "Misc", price: 10, stock: 2 })
db.products.deleteMany({ stock: { $lt: 5 } })
```

```
storeDB> db.products.insertOne({ name: "Old Item", category: "Misc", price: 10, stock: 2 })
| db.products.deleteMany({ stock: { $lt: 5 } })
{ acknowledged: true, deletedCount: 1 }
```

11. Count total remaining documents in the collection using countDocuments().

```
db.products.countDocuments()
```

```
storeDB> db.products.countDocuments()
3
```

12. Sort results by price descending and limit to top 2 most expensive products.

```
db.products.find().sort({ price: -1 }).limit(2)
```

```
storeDB> db.products.find().sort({ price: -1 }).limit(2)
[
  {
    _id: ObjectId('6ac14122ba47f868f297b4a5'),
    name: 'Laptop',
    category: 'Electronics',
    price: 55000,
    stock: 22
  },
  {
    _id: ObjectId('6ac14122ba47f868f297b4a7'),
    name: 'Desk Chair',
    category: 'Furniture',
    price: 4500,
    stock: 20
  }
]
```

Exercise :

Volkswagen Database (volkswagenDB)

1. Switch Database

```
test> use volkswagenDB
switched to db volkswagenDB
```

2. Create (Insert Data) Add multiple new cars to the inventory.

```
volkswagenDB> db.cars.insertMany([
|   {
|     model: 'Taigun',
|     trim: 'Highline',
|     engine: '1.0L TSI',
|     fuelType: 'Petrol',
|     transmission: '6-Speed AT',
|     year: 2024,
|     priceINR: 1580000,
|     inStock: true
|   },
|   {
|     model: 'Tiguan',
|     trim: 'Elegance',
|     engine: '2.0L TSI',
|     fuelType: 'Petrol',
|     transmission: '7-Speed DSG',
|     year: 2024,
|     priceINR: 3517000,
|     inStock: false
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac1470d7c5d7a8e3322e051'),
    '1': ObjectId('6ac1470d7c5d7a8e3322e052')
  }
}
```

3. Read (Retrieve Data) Find all cars that are currently in stock.

```
volkswagenDB> db.cars.find({ inStock: true }).pretty()
[
  {
    _id: ObjectId('6abb3ceabc11f5625916551c'),
    model: 'Virtus',
    trim: 'GT Plus',
    engine: '1.5L TSI',
    fuelType: 'Petrol',
    transmission: '7-Speed DSG',
    year: 2024,
    priceINR: 1940000,
    inStock: true
  },
  {
    _id: ObjectId('6ac1470d7c5d7a8e3322e051'),
    model: 'Taigun',
    trim: 'Highline',
    engine: '1.0L TSI',
    fuelType: 'Petrol',
    transmission: '6-Speed AT',
    year: 2024,
    priceINR: 1580000,
    inStock: true
  }
]
```

4. Update (Modify Data) Update the price of the 'Virtus' (the document you already had in the database).

```
volkswagenDB> db.cars.updateOne(
|   { model: 'Virtus' },
|   { $set: { priceINR: 1955000 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Delete (Remove Data) Remove the 'Tiguan' from the database.

```
volkswagenDB> db.cars.deleteOne({ model: 'Tiguan' })
{ acknowledged: true, deletedCount: 1 }
```

Healthcare Database (healthcareDB)
1. Switch Database

```
volkswagenDB> use healthcareDB
switched to db healthcareDB
```

2. Create (Insert Data) Add a new patient record.

```
healthcareDB> db.patients.insertOne({
|   patientId: 'MED-2024-002',
|   name: 'Priya Sharma',
|   age: 28,
|   gender: 'Female',
|   bloodGroup: 'B+',
|   department: 'Neurology',
|   attendingPhysician: 'Dr. Rao',
|   admissionDate: new Date(),
|   status: 'Admitted'
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ac147227c5d7a8e3322e053')
}
```

3. Read (Retrieve Data) Find all patients assigned to 'Cardiology'.

```
healthcareDB> db.patients.find({ department: 'Cardiology' }).pretty()
[
  {
    _id: ObjectId('6abb3cffbc11f5625916551d'),
    patientId: 'MED-2024-001',
    name: 'Arjun Verma',
    age: 34,
    gender: 'Male',
    bloodGroup: 'O+',
    department: 'Cardiology',
    attendingPhysician: 'Dr. Nair',
    admissionDate: ISODate('2026-09-29T04:22:23.252Z'),
    status: 'Admitted'
  }
]
```

4. Update (Modify Data) Update Arjun Verma's status to 'Discharged'.

```
healthcareDB> db.patients.updateOne(
|   { patientId: 'MED-2024-001' },
|   { $set: { status: 'Discharged' } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Delete (Remove Data) Remove Priya Sharma's record using her specific Patient ID.

```
healthcareDB> db.patients.deleteOne({ patientId: 'MED-2024-002' })
{ acknowledged: true, deletedCount: 1 }
healthcareDB>
```